

StableGuard

Technical Whitepaper

A modular, MEV-aware collateralized
stablecoin protocol

Contents

1. Abstract	2
2. Introduction and Design Goals	2
3. System Overview	2
4. The Price Oracle	4
5. Collateral and Custody	4
6. Risk Configuration	4
7. Core Position Flows	6
8. Liquidation	6
9. Repeg and Arbitrage	7
10. Governance and the Timelock	7
11. Rate Limiting and Abuse Resistance	7
12. Security Model and Threat Model	9
13. Protocol Invariants	9
14. Engineering Notes	9
15. Testing and Verification	9
16. Parameter Reference	11
17. Repository Layout	11
18. Glossary	11
19. References	12

1. Abstract

StableGuard is a modular, gas-optimized ERC-20 stablecoin protocol built with the Foundry toolchain. Its token, **SGD**, is a USD-denominated debt instrument minted against over-collateralized positions in native ETH or supported ERC-20 assets. The protocol decomposes the mechanics of a collateralized-debt stablecoin into seven single-responsibility contracts — a validating price oracle, a custody ledger, a risk-assessment module, a Dutch-auction engine, a peg-realignment coordinator, an arbitrage router, and a timelock — orchestrated by a single core contract that is the sole authority for minting, burning, and risk parameters.

This document describes the protocol as of **Version 2.0**, a substantial revision that closes a set of cross-contract coherence defects found in the original design. In the revised architecture, collateral custody is unified in one ledger with an authorized-mover model; every liquidation path settles debt at a single point that mints and burns SGD; token-to-USD conversion is decimal-aware and centralized in the oracle; risk thresholds have one source of truth; and governance-sensitive actions are routed through an enforced timelock. StableGuard is a research and portfolio reference: it runs deterministically against local mocks and a mainnet fork, ships with 443 passing tests including an end-to-end liquidation lifecycle, but it has not been audited and is not intended for production deployment.

2. Introduction and Design Goals

A collateralized-debt stablecoin must solve three problems at once: it must value heterogeneous collateral safely, it must retire bad debt before it becomes protocol insolvency, and it must keep the token near its target price under adversarial conditions. StableGuard treats each of these as a distinct concern with a distinct module, connected through explicit interfaces rather than shared mutable state. The result is a system that is easier to reason about, test in isolation, and extend.

Design goals

- **Single responsibility.** Each module owns exactly one concern — pricing, custody, risk math, auctioning, repeg, arbitrage, or delay — and holds no duties that belong to another.
- **One source of truth.** Risk parameters live only in the core contract's packed configuration; custody lives only in the collateral ledger; SGD is minted and burned only by the core.
- **Decimal-correct accounting.** All token↔USD conversion flows through the oracle and respects each asset's native decimals, so 6- and 8-decimal collateral is valued identically to 18-decimal.
- **Safety by construction.** Checks-Effects-Interactions ordering, reentrancy guards, an enforced timelock on governance, per-user and global rate limits, and conservative oracle fallbacks.
- **MEV awareness.** Auctions support a commit–reveal path, minimum bid spacing, price-impact caps, and flash-loan heuristics to reduce sandwiching and sniping.
- **Observability.** Every state transition emits an explicit event, and failure modes are predictable reverts, so the protocol can be monitored and post-mortemed off-chain.

3. System Overview

StableGuard is composed of eight deployed contracts and one linked library. The core contract, **StableGuard**, is simultaneously the ERC-20 token and the orchestrator: users interact almost exclusively with it, and it delegates specialized work to the surrounding modules while retaining exclusive authority over the SGD supply and the risk configuration.

Modules and responsibilities

Contract	Responsibility
StableGuard	Core ERC-20 (SGD). Orchestrates deposit/mint and burn/withdraw, holds the packed risk configuration (the single source of truth), and is the only contract that mints or burns SGD and the sole settlement point for all liquidation paths.
PriceOracle	Chainlink-aggregator validation with freshness and deviation guards; decimal-aware normalization to 18 decimals; conservative fallback prices. Exposes getTokenValueInUsd and getTokenAmountFromUsd.
CollateralManager	Sole custodian of user collateral (ETH and ERC-20). Only authorized managers may move custody; withdrawals debit the user and pay a named recipient directly.
LiquidationManager	Pure risk-assessment and conversion math. Reads thresholds from the core; never holds funds and never executes a liquidation itself.
DutchAuctionManager	Descending-price liquidation auctions. Bids are paid in SGD; a winning bid settles debt in the core and seizes collateral from the ledger. Commit–reveal and MEV protections.
RepegManager	Monitors deviation from the \$1 target and applies bounded buy/sell pressure under cooldowns and caps. Tracks ETH and SGD liquidity in separate pools.
ArbitrageManager	Routes guarded trades on Uniswap V2 to align DEX spot with the oracle, protected by a circuit breaker.
Timelock	Queue / execute / cancel lifecycle enforcing a minimum delay on the core's governance actions.
RateLimiter	External library (delegatecall-linked) implementing per-user and global rate limiting; keeps the core contract under the EIP-170 size limit.

Actors. A **borrower** deposits collateral and mints SGD; a **keeper** or **bidder** restores solvency by liquidating unsafe positions; the **owner** (ideally a multisig) performs emergency actions directly and routine parameter changes through the timelock; an **oracle provider** publishes the Chainlink feeds the protocol validates.

Deployment note — circular construction

`RepegManager` and `ArbitrageManager` require the SGD (core) address in their constructors, while the core requires their addresses in turn. The deployment script resolves this by predicting the core address with `vm.computeCreateAddress` and asserting the prediction after deployment, then wiring custody authorizations and module references.

4. The Price Oracle

Correct collateral valuation is the foundation of a lending stablecoin, and the two most common failure modes are decimal mishandling and stale or manipulated feeds. The oracle addresses both. Every Chainlink answer is converted to a common 18-decimal representation before any comparison, and all downstream valuation flows through two functions so that no consumer re-implements the math:

- `getTokenValueInUsd(token, amount)` — the USD value (18 decimals) of a token amount expressed in the token's own decimals.
- `getTokenAmountFromUsd(token, usdValue)` — the inverse: the token amount (native decimals) worth a given USD value. Used to size seized collateral in liquidations.

Validation and safety

On each read, the oracle applies a layered validation before accepting a Chainlink answer, and otherwise serves a stored fallback price rather than reverting the whole flow:

1. **Well-formedness.** The round id, answer sign, update time, and answered-in-round must be internally consistent.
2. **Freshness.** The answer must be no older than the staleness threshold (default 3,600 s / 1 hour), shared by both price reads and the health check so they cannot disagree.
3. **Dust floor.** After conversion to 18 decimals, the price must exceed `MIN_VALID_PRICE` (\$0.0001), rejecting degenerate near-zero feeds.
4. **Deviation guard.** A move larger than `MAX_PRICE_DEVIATION` (50%) from the last accepted price is rejected in favor of the fallback; a move above 10% is accepted but emits a monitoring event.

A single sudden 10x spike is therefore rejected and replaced by the fallback, while a genuine trend that arrives in sub-50% steps is tracked. The last-valid-price state that powers the deviation guard is written on every accepted read — a correctness fix over the prior version, where the update was a no-op and the guard was silently inert.

5. Collateral and Custody

All user collateral is held by a single contract, the `CollateralManager`. Centralizing custody means there is exactly one balance to reconcile and one place where collateral can move. Movement is gated by an authorized-manager model: the core and the auction engine are registered as authorized movers, and only they may call

```
withdraw(user, token, amount, recipient)
```

which debits the user's recorded custody and transfers the asset directly to a named recipient — a keeper, an auction winner, or the borrower themselves. This replaces the prior design, in which liquidation modules attempted to transfer collateral from their own (empty) balances and never decremented the user's custody, leaving accounting divergent and transfers reverting. Deposits are always routed through the core, which pulls the ERC-20 (or receives ETH) and credits the ledger, so a user's recorded balance and the ledger's actual holdings never drift apart.

6. Risk Configuration

Risk thresholds are defined once, in the core contract's packed configuration, and every other module reads them from there via `getSystemConfig()`. Consolidating these values eliminates the class of bug where the same threshold is duplicated across contracts and silently diverges after an update. The shipped defaults are:

Parameter	Default	Meaning
<code>minCollateralRatio</code>	150%	Minimum collateral-to-debt ratio required to mint or to remain healthy after an operation.
<code>liquidationThreshold</code>	120%	Below this ratio a position becomes liquidatable.
<code>emergencyThreshold</code>	110%	Distress band used by emergency tooling.
<code>maxLiquidationBonus</code>	10%	Incentive paid to liquidators, applied when sizing seized collateral.
<code>emergencyDelay</code>	3,600 s	Delay associated with emergency actions.

Invariant — solvency
$$\text{collateral_value_usd} \geq \text{debt_usd} \times (\text{minCollateralRatio} / 10000)$$

Enforced before every mint and re-checked after every operation that reduces collateral or increases debt. A position is liquidatable once its ratio falls below `liquidationThreshold`.

7. Core Position Flows

7.1 Deposit and mint

`depositAndMint(token, depositAmount, mintAmount)` opens or grows a position.

1. The oracle values the incoming collateral in USD (decimal-aware).
2. The prospective position — existing collateral plus the deposit, existing debt plus the mint — is checked against `minCollateralRatio`; the call reverts if it would be unsafe.
3. SGD is minted to the borrower and the debt is recorded.
4. The collateral is transferred to the `CollateralManager` and credited to the borrower.

7.2 Burn and withdraw

`burnAndWithdraw(token, burnAmount, withdrawAmount)` reduces or closes a position: SGD is burned, debt is decreased, and collateral is released — but only if the residual position remains at or above the minimum ratio, or the debt is fully repaid. Two guards matter here: the withdrawal is blocked while the collateral is committed to an active auction, and the collateral is paid to the borrower directly by the ledger rather than routed through the core.

8. Liquidation

When a position falls below the liquidation threshold, the protocol must retire its debt and distribute its collateral. StableGuard offers three settlement paths, all of which converge on a single core function that performs the actual debt reduction and SGD burn — the property that makes the accounting sound.

8.1 Dutch auction (primary path, settled in SGD)

A keeper calls `liquidate(user, debtAmount)` on the core, which asks the risk module for the optimal collateral token and opens a descending-price auction sized to cover the debt plus the liquidation bonus (never the borrower's entire balance). The auction price decays linearly from the start price to a floor of 50% over one hour. Bidding is paid **in SGD**, which is the key correction over the prior design, where bids were nonsensically denominated in the collateral being sold:

1. The bidder pays `currentPrice × collateralAmount` in SGD.
2. The debt portion — `min(totalCost, outstandingDebt)` — is forwarded to the core, which burns exactly that much SGD and reduces the borrower's debt.
3. Any surplus above the outstanding debt is paid to the liquidated borrower.
4. The seized collateral is withdrawn from the ledger straight to the winning bidder.
5. Settlement is atomic: if the core callback reverts, the entire bid reverts.

MEV protections. The engine supports a commit–reveal path (a 5-minute commit window and a 10-minute reveal window) so bids can be hidden until reveal, enforces a minimum 12-second delay between bids, caps per-auction price impact, and applies a simple multi-block flash-loan heuristic.

8.2 Direct liquidation

`liquidatePosition(user, token, debtAmount)` lets a keeper repay part of a position's debt in SGD and receive an equivalent amount of that collateral plus the bonus, priced through the oracle's `getTokenAmountFromUsd`. It is the fast path for orderly conditions.

8.3 Emergency liquidation

`emergencyLiquidate(user, token, debtAmount)` is an owner-only path in which the owner supplies the SGD being retired, so the supply stays fully backed: the SGD is pulled in, the debt is decremented once and burned once, and equivalent collateral is seized to the owner. This replaces a prior version that double-counted debt and double-burned via a swallowed callback.

9. Repeg and Arbitrage

Beyond collateral safety, the protocol includes optional machinery to defend the peg. The `RepegManager` monitors the deviation of SGD from its \$1 target and, when the deviation exceeds the configured threshold and cooldown and daily caps allow, applies bounded buy or sell pressure via Uniswap V2. A single direction convention is used consistently throughout: `1` means buy SGD (price below target, push it up) and `2` means sell SGD (price above target, push it down) — fixing a prior inconsistency in which the simulation and the liquidity accounting used opposite conventions.

- **Separate liquidity pools.** ETH and SGD liquidity are tracked independently, so a provider is always repaid in the asset they deposited; the caller of a successful reppeg receives the incentive (capped and configurable), not the core contract.
- **Bounded action.** The deviation threshold defaults to 5% with a hard cap of 20%; repeggs are rate-limited by a one-hour cooldown and a daily maximum, and incentives are capped at 100 SGD.
- **Arbitrage routing.** The `ArbitrageManager` executes guarded Uniswap V2 trades (max 100 ETH per trade, minimum 0.5% profit, 3% slippage bound) behind a circuit breaker that trips after three consecutive failures and recovers after one hour.

Accounting correctness

Realized arbitrage profit accrues to a per-address **reputation** score used only to grant a 10% incentive bonus — it is never a claimable balance, closing a prior defect where a notional profit was credited as if the contract held those funds.

10. Governance and the Timelock

Sensitive parameter changes cannot take effect instantly. The `Timelock` enforces a queue → wait → execute lifecycle, and the core contract's governance entry points — `updateModules`, `updateConfig`, and `updateRepegConfig` — are callable only by the timelock, not by the owner directly. This is a material change over the prior version, where the timelock was deployed but wired to nothing and all admin functions were plain owner calls.

Timelock parameter	Value
Minimum delay	2 days
Maximum delay	30 days
Grace period	14 days

Two-tier authority. Governance actions flow through the timelock; genuinely urgent responses — emergency liquidation, pausing the reppeg module, and rate-limit resets — remain direct owner calls so the protocol can react without waiting out a delay.

11. Rate Limiting and Abuse Resistance

Core user operations are throttled to blunt spam and griefing. The logic lives in the `RateLimiter` library, whose public functions are `delegatecall`-linked so that their bytecode sits outside the core contract (see §13). Limits are enforced per user and globally:

Limit	Value
Rolling window	1 hour
Max operations per user / hour	10
Max volume per user / hour	100,000 (USD-equivalent)
Per-operation cooldown	5 minutes
Burst limit / window	3 operations / 1 minute
Global operations per hour	1,000

12. Security Model and Threat Model

- **Access control.** OpenZeppelin `Ownable` on the peripheral modules, an immutable owner on the core and the timelock, an authorized-mover set on the custody ledger, and timelock-only governance.
- **Reentrancy.** Checks-Effects-Interactions ordering with reentrancy guards on state-mutating entry points; the settlement callback is guarded and atomic.
- **Oracle risk.** Freshness, dust-floor, and deviation validation with conservative fallback prices; the health check and the read path share one staleness threshold.
- **MEV / sniping.** Commit–reveal auctions, minimum bid spacing, price-impact caps, and flash-loan heuristics.
- **Parameter abuse.** Timelock delays and bounded parameter ranges prevent sudden unsafe changes.
- **Denial of service.** Per-user and global rate limits, plus the arbitrage circuit breaker.

Residual risks (honest disclosure)

- The protocol has **not been audited**; the work behind v2 was a coherence and correctness pass, not an adversarial security review.
- The repeg and arbitrage modules interact with a live Uniswap V2 router; their internal logic is tested but not exercised against a real router on a fork.
- The core contract retains hand-written assembly in hot paths; it is not memory-safe under the IR pipeline, so the project pins the legacy code generator.

13. Protocol Invariants

- **Solvency.** Every open position holds collateral worth at least its debt times the minimum ratio at mint time and after each collateral-reducing operation.
- **Supply integrity.** SGD is minted and burned only by the core; every liquidation path burns exactly the debt it settles, so supply tracks outstanding debt.
- **Custody conservation.** Collateral held by the ledger equals the sum of all users' recorded custody at all times.
- **Single-source config.** Every module's view of risk thresholds equals the core's packed configuration.
- **Governance delay.** Queued governance actions cannot execute before their eta, and expire after the grace period.

The last invariant is asserted directly by a Foundry invariant test that drives the timelock through a handler and fails if any queued transaction ever executes early.

14. Engineering Notes

- **Toolchain.** Solidity `^0.8.19`, Foundry, OpenZeppelin Contracts v5.4.0, forge-std v1.11.0. Optimizer enabled at 200 runs; the IR pipeline is intentionally disabled.
- **Contract size.** The core compiles to 24,347 bytes, within the 24,576-byte EIP-170 limit. The margin is created by extracting rate limiting into the delegatecall-linked `RateLimiter` library rather than by enabling the IR pipeline, which would alter the behavior of the core's assembly.
- **Storage discipline.** Configuration, modules, and positions are packed; hot-path readers use assembly, and the layout is deliberately never reordered.
- **Deployment.** A single Foundry script deploys and wires all contracts, predicting the core address to satisfy the circular constructor dependency and asserting the prediction before proceeding.

15. Testing and Verification

The protocol ships with 443 tests across 15 suites, all passing. Unit suites cover each module against deterministic mocks; a dedicated integration suite wires the real contracts exactly as the deployment script does and exercises the full lifecycle — deposit, mint, an oracle price drop, and settlement through each of the

three liquidation paths — using an 8-decimal collateral token to catch decimal errors. Verification also includes a timelock invariant, a clean-room build, and a deploy dry-run against a mainnet fork.

```
forge build forge test -vv forge script script/Deploy.s.sol --fork-url <rpc>
```

16. Parameter Reference

Consolidated view of the shipped defaults. Basis points (bps) are hundredths of a percent; 10,000 bps = 100%.

Domain	Parameter	Value
Collateral	minCollateralRatio	15,000 bps (150%)
Collateral	liquidationThreshold	12,000 bps (120%)
Collateral	emergencyThreshold	11,000 bps (110%)
Collateral	maxLiquidationBonus	1,000 bps (10%)
Oracle	staleness threshold	3,600 s (1 h)
Oracle	min valid price	1e14 (\$0.0001)
Oracle	max deviation → fallback	5,000 bps (50%)
Auction	duration / price floor	3,600 s / 50% of start
Auction	commit / reveal window	300 s / 600 s
Auction	min bid delay	12 s
Repeg	deviation threshold / cap	500 bps (5%) / 2,000 bps (20%)
Repeg	cooldown / daily max	3,600 s / 10
Repeg	max incentive	100 SGD
Arbitrage	max trade / min profit	100 ETH / 50 bps (0.5%)
Arbitrage	max slippage / cooldown	300 bps (3%) / 60 s
Rate limit	ops / volume per hour	10 / 100,000
Rate limit	cooldown / burst	5 min / 3 per min
Timelock	min / max / grace	2 d / 30 d / 14 d

17. Repository Layout

```
src/
  StableGuard.sol      PriceOracle.sol      CollateralManager.sol
  LiquidationManager.sol DutchAuctionManager.sol RepegManager.sol
  ArbitrageManager.sol Timelock.sol RateLimiter.sol Constants.sol
  interfaces/ (IPriceOracle, ICollateralManager, ILiquidationManager,
              IDutchAuctionManager, IArbitrageManager, IRepegManager,
              IStableGuard, AggregatorV3Interface)
script/Deploy.s.sol
test/ (one suite per module + Integration.t.sol)
```

18. Glossary

- **SGD** — the StableGuard USD-denominated stablecoin token.
- **CDP** — collateralized debt position; collateral locked against minted stablecoin.
- **Collateral ratio** — collateral value divided by debt value, in basis points.
- **Basis point (bps)** — one hundredth of a percent; 10,000 bps = 100%.
- **Commit–reveal** — a two-phase bid scheme that hides bids until a reveal window to reduce MEV.
- **Fallback price** — a conservative stored price served when a live feed fails validation.
- **Timelock** — an enforced delay between queuing and executing a sensitive action.

19. References

- Foundry — Ethereum development and testing toolchain.
- OpenZeppelin Contracts v5.4.0 — ERC-20, Ownable, ReentrancyGuard.
- Chainlink — aggregator (AggregatorV3) price-feed interface and normalization practice.
- Uniswap V2 — automated-market-maker router used for repeg and arbitrage routing.
- EIP-170 — 24,576-byte contract-size limit motivating library extraction.

Disclaimer. StableGuard is a research and portfolio reference implementation. It has not been audited and is not intended for production or mainnet deployment with real value. Risk parameters are illustrative and must be re-evaluated for any real use. © 2026 Martín Casais, released under the MIT License.